Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 3

За седьмой семестр

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Группы ИИ-26

Згера Е. А.

Проверил:

Хацкевич А.С.

2026

Вариант 5

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Задачи лабораторной работы

Порядок выполнения лабораторной работы (общий для всех, целевые параметры и используемые векторы зависят от индивидуального варианта (см. таблицу вариантов)).

Задание 1. Базовый перехват запуска процессов

1. Разработайте eBPF-зонд (C + Python), использующий механизм перехвата kprobe (или tracepoint) для отслеживания системного вызова execve (запуск новых процессов).
2. Напишите логику на стороне ядра (C), которая извлекает идентификатор процесса (PID), идентификатор пользователя (UID) и имя исполняемого файла (comm).
3. Запустите скрипт с правами root и продемонстрируйте перехват запуска стандартных утилит (например, ls или cat) в соседнем окне терминала или новой вкладке WSL.

Задание 2. Динамический анализ аномального поведения (Индивидуальный вариант)

1. Определите целевой системный вызов и условие аномального поведения согласно вашему индивидуальному варианту.
2. Напишите eBPF-зонд на C, извлекающий аргументы выбранного системного вызова (например, путь к целевому файлу, целевой PID или сетевой порт).
3. Реализуйте логику фильтрации **строго на стороне ядра**: программа должна передавать событие в пространство пользователя (через кольцевой буфер BPF_PERF_OUTPUT) *только* при совпадении с условием аномалии. Передача всех подряд событий в Python-скрипт является грубой ошибкой производительности.
4. Сгенерируйте целевую аномалию (например, попытайтесь прочитать защищенный файл) и зафиксируйте успешный перехват алерта вашим скриптом.

Задание 3. Интеграция с пространством пользователя и аудит

1. Расширьте Python-скрипт из задания 2: при получении события об аномалии из ядра скрипт должен автоматически записывать предупреждение в локальный лог-файл security_audit.log.
2. Формат записи должен содержать: временную метку (Timestamp), PID нарушителя, UID нарушителя и детали события.

Задание 4. Проектирование профилей защиты (Без написания кода)

1. Сравните механизмы изоляции Seccomp-BPF и eBPF (kprobes) применительно к задаче блокировки системных вызовов. Обоснуйте, почему стандартный eBPF-зонд преимущественно используется для мониторинга и аудита, а не для предотвращения (блокировки) действий вредоносного ПО.
2. Опишите концепцию атаки состояния гонки типа TOCTOU (Time-of-Check to Time-of-Use) применительно к вашему eBPF-зонду из задания 2 и предложите архитектурный метод противодействия.

5	sys_enter_unlinkat	Попытка удаления файлов в системной директории /var/log/

Задание 1 Базовый перехват запуска процессов

Цель: разработать eBPF-зонд (C + Python), использующий механизм kprobe для отслеживания системного вызова execve (запуск новых процессов), извлекающий идентификатор процесса (PID), идентификатор пользователя (UID) и имя исполняемого файла (comm).

Листинг eBPF-программы (Python-обертка и C-код для ядра):

```
#!/usr/bin/python3
from bcc import BPF
import time

bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>
struct data_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
};
BPF_PERF_OUTPUT(events);

int syscall__execve(struct pt_regs *ctx) {
    struct data_t data = {};

    u64 id = bpf_get_current_pid_tgid();
    data.pid = id >> 32;

    data.uid = bpf_get_current_uid_gid() >> 32;

    bpf_get_current_comm(&data.comm, sizeof(data.comm));

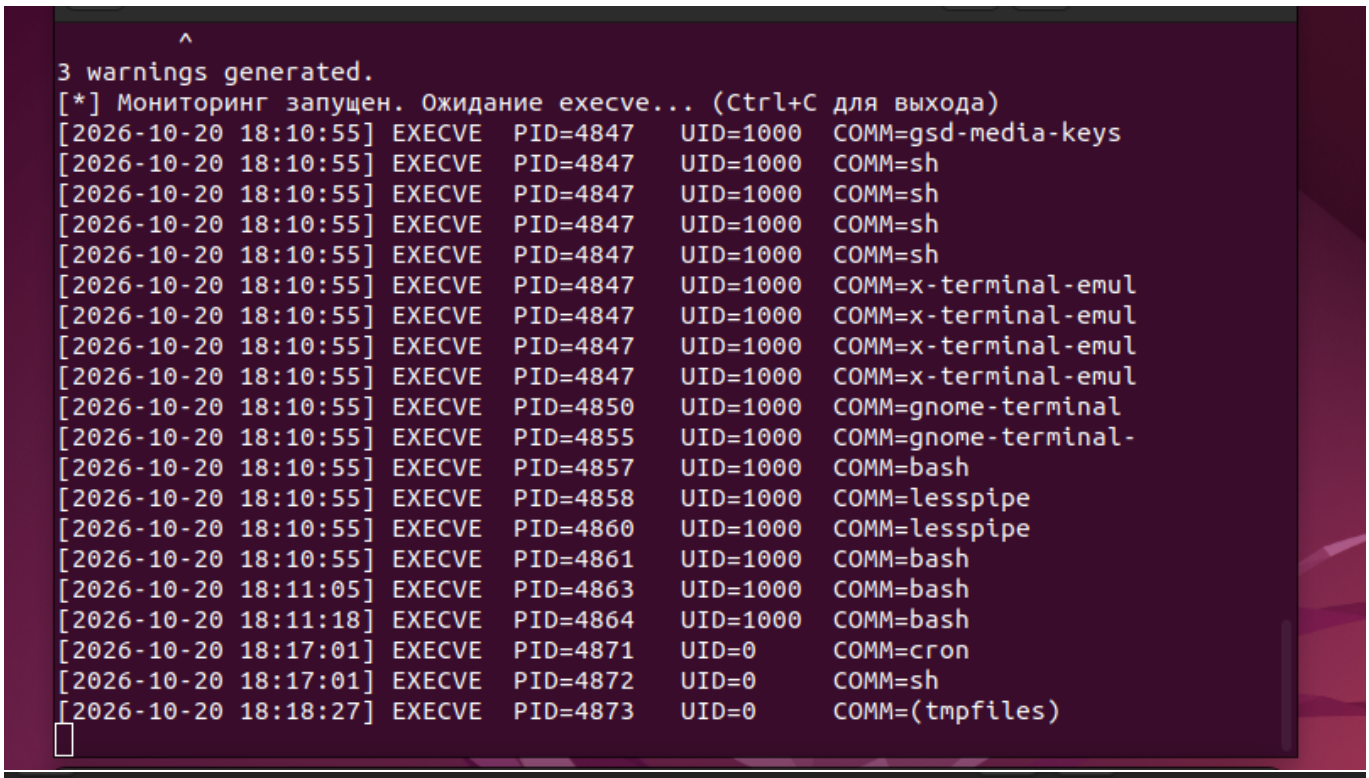
    events.perf_submit(ctx, &data, sizeof(data));
    return 0;
}
"""
b = BPF(text=bpf_text)
execve_fnname = b.get_syscall_fnname("execve")
b.attach_kprobe(event=execve_fnname, fn_name="syscall__execve")

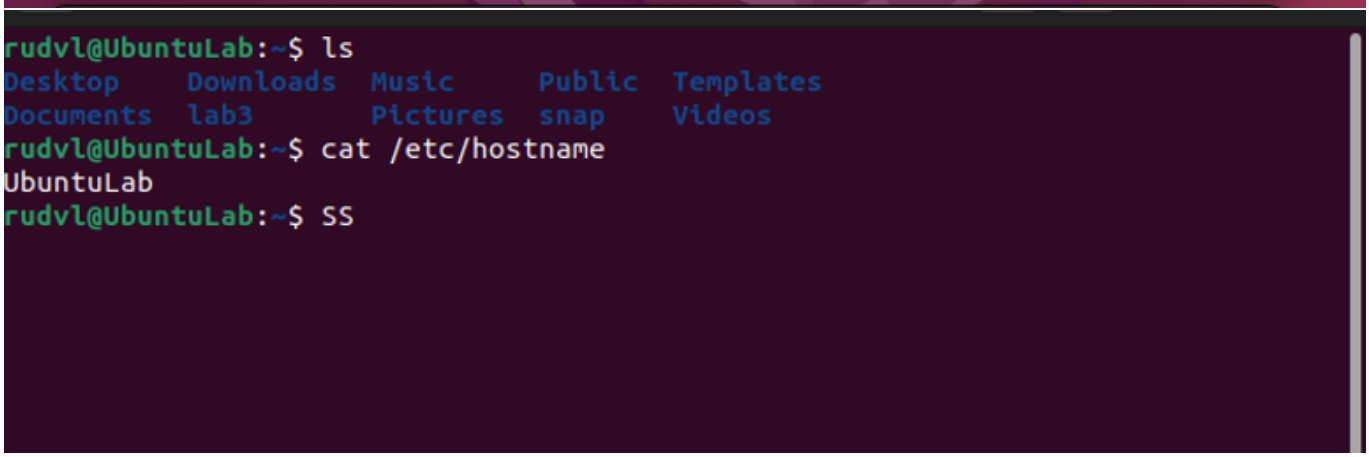
print("[*] Мониторинг запущен. Ожидание execve... (Ctrl+C для выхода)")

def print_event(cpu, data, size):
    event = b["events"].event(data)
    comm = event.comm.decode("utf-8", "replace")
    ts = time.strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{ts}] EXECVE  PID={event.pid:<6} UID={event.uid:<5} COMM={comm}")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```





Задание 2 Динамический анализ аномального поведения

Условие индивидуального варианта (Вариант №5):

Условие индивидуального варианта (Вариант №5):

Целевой системный вызов — sys_enter_unlinkat (удаление файлов). Условие аномального поведения — попытка удаления файлов в системной директории /var/log/. Фильтрация выполняется строго на стороне ядра.

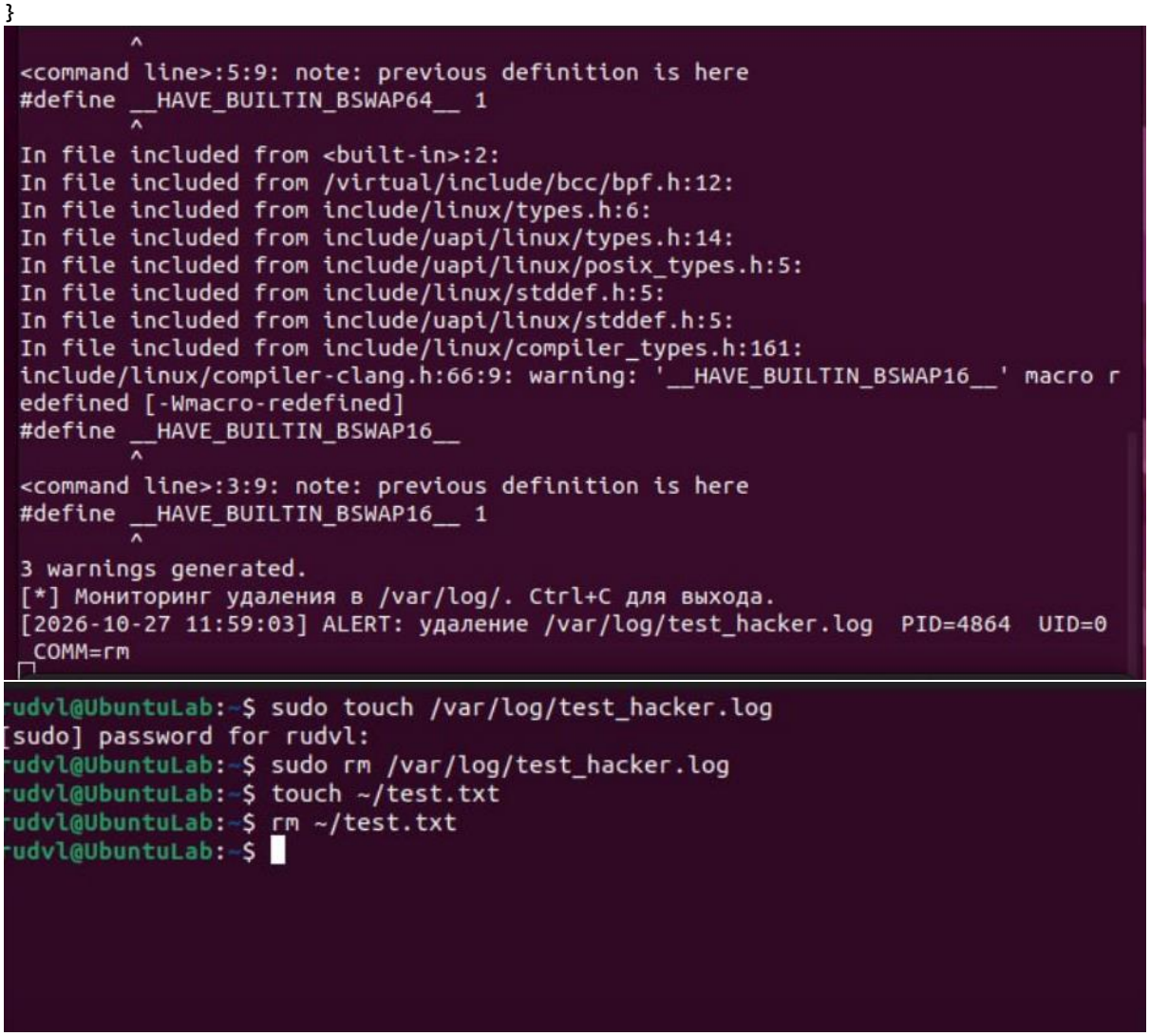
Листинг С-кода зонда (с выделенной логикой фильтрации):

```
TRACEPOINT_PROBE(syscalls, sys_enter_unlinkat) {
    struct data_t data = {};
    char path[64];

    long ret = bpf_probe_read_user_str(path, sizeof(path), args->pathname);
    if (ret <= 0)
        return 0;
    const char target[] = "/var/log/";
    #pragma unroll
    for (int i = 0; i < sizeof(target) - 1; i++) {
        if (path[i] != target[i])
            return 0; // Легитимные вызовы отсекаются здесь и НЕ уходят в Python
    }
    u64 id = bpf_get_current_pid_tgid();
    data.pid = id >> 32;
```

```
    data.uid = bpf_get_current_uid_gid() >> 32;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    __builtin_memcpy(data.filename, path, sizeof(data.filename));

    events.perf_submit(args, &data, sizeof(data)); // Отправка только при аномалии
    return 0;
}
```



Задание 3. Интеграция с пространством пользователя и аудит

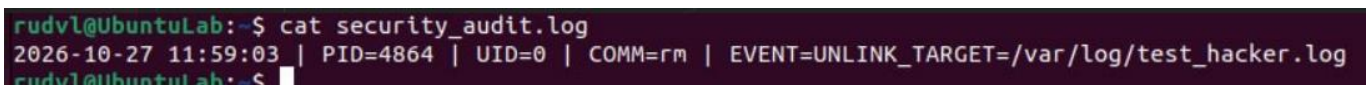
Фрагмент Python-скрипта, реализующий запись алертов в файл security_audit.log:

```
def print_event(cpu, data, size):
    event = b["events"].event(data)
    comm = event.comm.decode("utf-8", "replace")
    fname = event.filename.decode("utf-8", "replace")
    ts = time.strftime("%Y-%m-%d %H:%M:%S")

    print(f"[{ts}] ALERT: удаление {fname}  PID={event.pid}  UID={event.uid}  COMM={comm}")

    # Запись аудита в локальный файл
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(f"{ts} | PID={event.pid} | UID={event.uid} | COMM={comm} |
EVENT=UNLINK_TARGET={fname}\n")
```

Содержимое файла security_audit.log**:**



Задание 4. Проектирование профилей защиты

1. Сравнение механизмов Seccomp-BPF **и** eBPF (kprobes) **применительно к блокировке системных вызовов.**
Seccomp-BPF — это механизм принудительного ограничения, который работает как «белый/черный список» системных вызовов. Он проверяет номер вызова и его аргументы *до* выполнения и может заблокировать вызов (вернув SECCOMP_RET_ERRNO) или убить процесс (SECCOMP_RET_KILL). Он изначально спроектирован для предотвращения действий.
eBPF kprobes — это, прежде всего, механизм трассировки (мониторинга). Программа eBPF прикрепляется к точке входа в функцию ядра и выполняется уже *после* того, как системный вызов начал выполняться. Прерывание системного вызова из kprobe (например, через bpf_override_return) может привести к нестабильности ОС, так как ядро останется в несогласованном состоянии (могут не освободиться ресурсы, не сняться блокировки). Поэтому стандартный eBPF-зонд используется для мониторинга и аудита, а для блокировки (enforcement) используются Seccomp-BPF или eBPF LSM (Linux Security Module).
2. Концепция атаки TOCTOU (Time-of-Check to Time-of-Use) и метод противодействия.
В нашем зонде из Задания 2 eBPF-программа копирует путь pathname из пользовательского пространства в момент входа в системный вызов unlinkat (Time-of-Check) и проверяет, начинается ли он с /var/log/. Между моментом проверки и моментом, когда ядро фактически использует этот указатель для удаления файла (Time-of-Use), злоумышленник может изменить содержимое памяти по этому указателю (например, из параллельного потока). В результате eBPF «увидит» безопасный путь, а ядро удалит файл в /var/log/, и аудит будет обойден.
Архитектурный метод противодействия: перенос логики контроля на LSM-хуки (например, path_unlink или inode_unlink). Такие хуки срабатывают в момент фактического выполнения операции, когда ядро уже разрешило путь и получило struct path * / struct inode *, поэтому подмена строки в пользовательской памяти на них не влияет. Безопасное копирование памяти через bpf_probe_read_user_str защищает от паники ядра при «плохом» указателе, но не устраняет уязвимость TOCTOU.

Заключение

В ходе выполнения этой лабораторной работы мы научились использовать технологию eBPF, чтобы «заглянуть внутрь» ядра Linux и следить за его работой.
Сначала мы написали простую программу-монитор, которая перехватывала запуск любых новых процессов (системный вызов execve) и показывала, кто именно запустился: его PID, UID и имя программы.
Затем мы сделали более сложный и полезный для безопасности скрипт. По нашему варианту (№5) он следит за попытками удалить файлы (unlinkat), но главная фишка в том, что фильтрация происходит прямо внутри ядра. Программа в ядре проверяет, что путь начинается с /var/log/, и отправляет нам сигнал только в этом случае. Мы убедились, что если удалять файлы в других директориях, система молчит и не нагружает наш Python-скрипт лишней работой. Также мы научили программу записывать все такие подозрительные попытки в файл-аудит security_audit.log, чтобы у администратора оставались «логи» для разбора полетов.
В теоретической части мы поняли, почему такие eBPF-шпионы нельзя использовать для того, чтобы просто взять и заблокировать вирус. Если оборвать системный вызов на полпути, можно уронить (сломать) всю операционную систему. Для запретов есть другие инструменты (например, Seccomp). А еще мы изучили хитрую атаку TOCTOU: когда хакер меняет путь к файлу в самый последний момент, чтобы обмануть проверку.
Отдельно хочется отметить, что запуск eBPF в WSL2 на Windows — это испытание, так как там не хватает системных файлов ядра. Поэтому работа выполнялась в полноценной виртуальной машине с Ubuntu 22.04, где всё заработало стабильно.